

Renode Lab on GitHub Codespaces

Fourteen labs from a 5-min ARM MMIO warm-up to a custom C# peripheral and optional capstones up to a Coral NPU transformer robot — emulated entirely in your browser.

Welcome. Over the next few hours you will simulate fourteen different embedded systems — from a 5-minute MMIO taste on a bare-metal ARM Cortex-A9, through a Cortex-M4 microcontroller and a multi-core RISC-V Linux SoC, all the way to a SiFive FE310 with timer interrupts, a Robot-driven CI suite, a peripheral you write yourself in C#, optional capstones (cloud IoT, an edge-AI robot controller, Transformer robot using Coral NPU) — all inside a browser tab, with **nothing installed on your laptop**. This document walks you through the setup once. After that, everything you need is in the per-lab `README.md` files in the repository.

1. What you need before starting

1.1 A personal GitHub account

If you don't already have one, create a **free personal account** at <https://github.com/join>. School-issued accounts work too, but a personal account is preferred so the Codespace and any edits you make follow you after the course ends.

GitHub's free tier includes:

Resource	Free quota / month
Codespaces compute	120 core-hours ($\approx$ 60 hours on a 2-core machine)
Codespaces storage	15 GB-month

The whole renode-lab fits comfortably in those limits if you **stop your Codespace** when you take a break and **delete it** when you're done for the day. Leaving a Codespace running idle burns compute hours; leaving it created-but-stopped burns storage. We'll cover both at the end.

1.2 (Recommended-but optional for this lab) The GitHub Student Developer Pack

If you're a student, apply for the free **GitHub Student Developer Pack**. It upgrades your account to **GitHub Pro** at no cost, which **roughly doubles your Codespaces quota** — a real benefit for this lab and every future one you run in Codespaces:

Resource	Free account	With Student Pack (Pro)
Codespaces compute	120 core-hours / month	180 core-hours / month
Codespaces storage	15 GB-month	20 GB-month

(Quotas are GitHub's published figures at the time of writing and may change; check your account's Billing page for the current numbers.)

The Pack also bundles dozens of other free developer tools and cloud credits you'll find useful beyond this course.

How to get it (do this before your first launch if you can — approval can take anywhere from a few minutes to a few days):

1. Make sure you're signed in to the **personal GitHub account** from §1.1 (apply on the account you'll actually use for the labs).
2. Go to <https://education.github.com/pack> and click **Sign up for Student Developer Pack** (then **Get student benefits**).
3. Choose your account type as **Student** and select your school.
4. **Verify you're a student.** Fastest is a **school-issued email address** (@school.edu) — GitHub often approves it instantly. If you don't have one, upload **dated proof of enrollment** instead (student ID, class schedule, transcript, or enrollment letter with your name and school name visible).
5. Add your school name and **expected graduation date**, then submit.
6. Watch for the approval email. Once approved, **GitHub Pro is applied to your account automatically** — there's nothing to install, and your higher Codespaces quota takes effect immediately.

Already have it? If your account already shows GitHub Pro (Student), you're done — just launch the lab in §2. **Not a student, or waiting on approval?** The free-tier quota in §1.1 is enough to complete every lab; the Pack simply gives you more headroom.

1.3 A modern web browser

Chrome, Firefox, Safari, or Edge — anything from the last two years. You will spend the entire lab in two browser tabs:

1. The **VS Code** tab that GitHub auto-opens (your terminal + editor live here).
2. The **noVNC desktop** tab on port 6080 (Renode's GUI analyzers — UART windows, GPIO/LED indicators, etc.).

1.4 Working knowledge of basic Linux

You should be comfortable doing all of these from a terminal without looking them up:

- Navigating: `pwd`, `cd`, `ls`, `ls -la`
- Reading files: `cat`, `less`, `head`, `tail`, `tail -f`
- Editing files: `nano`, `vim`, or just VS Code's editor pane
- Pipes and redirection: `|`, `>`, `>>`, `2>&1`
- Background jobs: `&`, `Ctrl-C`, `Ctrl-Z`, `jobs`, `fg`
- Inspecting processes: `ps`, `pgrep -af <name>`
- Reading exit codes and recognising errors

You do **not** need to know Docker, Kubernetes, or anything about Codespaces internals.

1.5 Familiarity with basic SoC concepts

You should already understand, at a hand-wave level:

- A CPU **fetches** instructions from memory addresses, then **executes** them. The address it's about to fetch from is the **program counter (PC)**.
- A **bus** lets the CPU talk to memory regions and **memory-mapped peripherals** (UART, GPIO, timers, interrupt controller, etc.). Each peripheral lives at a known base address.
- A **UART** sends bytes one wire at a time; on most cores there's a "transmit holding register" and a "line status register" with a `TX-FIFO-empty` bit.
- A **GPIO port** has registers for direction (input/output), output data (`ODR`), and atomic set/reset (`BSRR` on STM32).
- An **ELF** is a packaged binary with a known **entry point**; on Cortex-M, after reset the CPU loads SP from address `0x0` and PC from address `0x4` (the vector table).
- A **bootloader** (BBL, OpenSBI, U-Boot) initialises the SoC enough to hand control to a kernel.
- A **kernel** schedules processes; a **userspace shell** like BusyBox is the program you type commands into.

Lab 01 will exercise the first four bullets, lab 02 the last three, and lab 03 lets you build the bus + RAM + UART picture from nine lines of declarative platform description.

2. First launch (only do this once)

2.1 Open the lab repository

Visit <https://github.com/prag79/renode-lab>. Sign in to your GitHub account if prompted.

2.2 Click the “Open in GitHub Codespaces” badge

Near the top of the page you’ll see this badge:



Click it. GitHub will:

1. Show a “**Create codespace**” screen — accept the defaults (2-core machine, 8 GB RAM, 32 GB storage). Click **Create codespace**.
2. Spin up a small Linux VM somewhere in their cloud (≈ 30 s).
3. Pull the prebuilt image `ghcr.io/prag79/renode-lab:latest` from the GitHub Container Registry (≈ 30 – 60 s, only on first launch — afterwards it’s cached).
4. Open VS Code in your browser, attached to the running container.

2.3 Verify the environment

In the VS Code terminal (open one with **Ctrl-`** if it isn’t already), type:

```
lab list
```

You should see the available labs listed (00–10 core, 11–13 optional capstones):

Lab	What it is
00	bare-metal ARM Cortex-A9 + SmartTimer MMIO demo (also: <code>lab demo</code>)
01	bundled STM32F4 demo (sanity check)
02	Linux on SiFive HiFive Unleashed (RISC-V)
03	custom RV64 SoC + bare-metal hello world
04	bare-metal on a SiFive FE310 (HiFive1): UART + GPIO blink
05	FE310 timer interrupts: blink from a CLINT ISR
06	headless regression testing with the Robot framework
07	model your own peripheral (custom C# timer IP)
08	multi-node IoT network: 3x FE310 over a shared UART bus
09	edge AI: run an int8 neural net on a bare-metal RISC-V core
10	real TensorFlow Lite Micro: gesture recognition on LiteX/VexRiscv
11	(optional) cloud IoT: stream JSON telemetry to AWS IoT Core / Azure IoT Hub
12	(optional) edge-AI robot: on-device int8 model turns commands into robot skill plans
13	(optional) transformer robot: language policy on a Google Coral NPU (<code>CPU.CoralNPU</code>); needs Renode NIGHTLY

Lab	What it is
monitor	plain Renode interactive monitor

Labs **11–13** are optional capstones — cloud telemetry, an on-device robot controller, and a transformer offloaded to a Google Coral NPU. You don't need them for the sanity check below; run them after labs 00–10 when you're ready.

Then run lab 01:

```
lab 01
```

You should land at a `(STM32F4_Discovery)` prompt within a couple of seconds. That's the **Renode monitor** — Renode's interactive command line. From here, type:

```
start
sysbus.cpu PC
sysbus.cpu PC
```

If the second `sysbus.cpu PC` prints a different value than the first, your environment is healthy. Type `quit` to exit.

If anything above fails, see § 5 **Troubleshooting**.

3. Where your edits live

The `lab NN` command does not run the lab from `/labs/...` directly. On first invocation it copies the lab into a **writable scratch tree** under your home directory and runs it from there:

Path	What it is	Stop?	Rebuild?	Delete?
<code>/labs/<lab-name>/</code>	Read-only image content	yes	no	no
<code>~/work/<lab-name>/</code>	Editable copy made by <code>lab NN</code>	yes	no	no
<code>/workspaces/renode-lab/</code>	The cloned git repo	yes	yes	only if pushed

Stop / Rebuild / Delete are the three Codespace lifecycle events: stopping the Codespace, rebuilding its container, and deleting it.

Practical rules:

- If you just want to tweak a `.resc` or a `hello.c` and re-run, edit `~/work/<lab-name>/...`. The next `lab NN` reuses your edits (`cp -ru` is idempotent — it never overwrites existing files).
- If you want changes to **outlive Codespace deletion**, you must push them to a git remote you own — see § 4.1 below. A **pull request is not required just to keep your work**; it's only needed if you want your changes merged into upstream.

3.1 Persisting your work past Codespace deletion

`prag79/renode-lab` is read-only for you (only the maintainer can push). To save your edits, fork the repo and push to your fork. Run these commands **inside the Codespace**, in the cloned repo at `/workspaces/renode-lab/`. Step 1 is one-time; steps 2–4 are the loop you'll run any time you want to checkpoint progress.

```
cd /workspaces/renode-lab
```

1. One-time: fork on GitHub and rewire the remotes. The `gh repo fork --remote` flag automatically renames the existing `origin` (which points at `prag79/renode-lab`) to `upstream` and adds your fork as the new `origin`:

```
gh repo fork --remote --remote-name=origin
git remote -v          # sanity check:
                        #   origin    https://github.com/<your-username>/renode-lab.git
                        #   upstream  https://github.com/prag79/renode-lab.git
```

2. (Recommended) work on a branch, not on main. This keeps your tinkering separate from upstream so future `git pull upstream main` is conflict-free:

```
git checkout -b my-experiments
```

3. Copy your edits from `~/work/...` into the git-tracked lab folders inside the cloned repo — not into `/labs/...` at the filesystem root (that path is read-only image content and is not in git):

```
cp -ru ~/work/00-Demo/.      /workspaces/renode-lab/labs/00-Demo/
cp -ru ~/work/03-custom-soc/. /workspaces/renode-lab/labs/03-custom-soc/
# ...repeat for any other lab you tweaked.
```

Two different labs paths — don't mix them up: - `/labs/00-Demo/` — baked into the Docker image, read-only, **not** in git. - `/workspaces/renode-lab/labs/00-Demo/` — inside your cloned repo, **tracked by git**. Copy here before `git commit`.

4. Commit and push to your fork.

```
git add -A
git status                                # review what changed
git commit -m "lab 03: my UART experiments"
git push -u origin my-experiments        # pushes to <you>/renode-lab
```

Visit <https://github.com/<your-username>/renode-lab/tree/my-experiments> and confirm your files are there. The Codespace itself is now disposable: `gh codespace delete -c <name>` is safe.

Get your work back on a fresh laptop / new Codespace. Open a new Codespace **from your fork** (on `<you>/renode-lab` click *Code* → *Codespaces* → *Create on my-experiments*) and your edits are already in `/workspaces/renode-lab/`.

Optional — contribute upstream. If you fixed a typo or want to share a lab improvement with everyone, open a PR:

```
gh pr create --repo prag79/renode-lab \
  --base main --head "$(gh api user -q .login):my-experiments" \
  --title "lab 03: my UART experiments" \
  --body "Short description of what changed and why."
```

4. Troubleshooting

Symptom	Likely cause	Fix
lab 01 prompt appears but sysbus.cpu PC keeps returning 0x00000000	You haven't typed start yet — the bundled .resc loads the ELF but doesn't release the CPU from reset	Type start once.

Symptom	Likely cause	Fix
The terminal is unreadable: UART log lines stream non-stop and your typing doesn't echo	UART output is being mirrored to the console at INFO level	pause (type blindly, then Enter), then <code>logLevel 3 sysbus.uart4</code> , then <code>start</code> .
noVNC tab shows "Failed to connect" or HTTP 502	Port 6080's WebSocket handshake hadn't completed when the tab opened	Wait 10 s and click Connect in the noVNC page. If still broken: in VS Code's Ports panel, right-click port 6080 → Port Visibility → Public (or sign in to the popup).
Browser tab opens but shows the noVNC welcome screen, not a desktop	URL is missing the path	Append <code>/vnc.html</code> to the URL, then Connect .
<code>pgrep -af 'Xvfb\ fluxbox\ x11vnc\ websockify'</code> shows fewer than four processes	The headless desktop didn't fully start	<code>/usr/local/bin/Run entrypoint.sh true</code> . Check the per-service logs in <code>/tmp/Xvfb_:1.log</code> , <code>/tmp/fluxbox.log</code> , <code>/tmp/x11vnc_-display_:1.log</code> , <code>/tmp/websockify.log</code> .
<code>lab list</code> stops before the latest optional labs after rebuild	Codespaces reused a cached <code>ghcr.io/...:latest</code> image even though CI pushed a newer one	<code>cd /workspaces/renode-lab && git pull</code> , then Rebuild Container (postStart now runs <code>sync-labs.sh</code> to overlay lab + labs/ from the repo). Or run <code>bash sync-labs.sh</code> by hand after <code>git pull</code> .
<code>lab</code> does nothing or says command not found	You're in a shell where <code>/usr/local/bin</code> isn't on <code>\$PATH</code> (rare)	Run it with the full path: <code>/usr/local/bin/lab list</code> .
Codespace fails to start with "Failed to pull image"	GHCR rate-limited you or the image package is private	Visit https://github.com/prag79?tab=packages → <code>renode-lab</code> → check it's Public . Retry.
You changed something under <code>/labs/...</code> , ran <code>lab NN</code> , and your change had no effect	Edits to <code>/labs/...</code> are overlaid on a read-only image and discarded on container rebuild — the dispatcher uses <code>~/work/<lab-name>/...</code>	Edit the file under <code>~/work/<lab-name>/...</code> instead, then <code>lab NN</code> again.

5. Cost discipline (so you don't burn quota)

GitHub will not charge you anything as long as you stay inside the free tier. Two habits keep you there:

1. **Stop the Codespace when you walk away.** In the bottom-left of VS Code (or the **Codespaces** menu in your GitHub profile) click **Stop codespace**. Compute billing pauses immediately. Storage keeps ticking at ~\$0.07 / GB / month — tiny, but cumulative.
2. **Delete the Codespace when you're done for the day** — but only after committing or copying out anything you want to keep. From your laptop terminal:

```
gh codespace list                                # find the name
gh codespace delete --codespace <name> --force
```

...or use the web UI at <https://github.com/codespaces>.

You can always recreate a Codespace from the badge in the README. The first launch is the only slow one (~60 s); after that it's instant.

6. Where to go after the labs

- The Renode user docs: <https://renode.readthedocs.io/>
- Renode's bundled platform `.repl` files for ~100 other boards: `/opt/renode/platforms/` inside your Codespace
- The Antmicro YouTube channel for SoC-modelling walkthroughs.

If you find a bug or have an improvement, open a PR against <https://github.com/prag79/renode-lab>. The full toolchain (lab content → Docker image → GHCR → Codespaces) is in this one repository and welcomes contributions.

Quick reference card (a single page you can print as a cheat sheet):

Task	Command / how to do it
Open the lab	click the Open in GitHub Codespaces badge in the README
List labs	<code>lab list</code> (the banner runs this on attach)
Quick taste	<code>lab 00</code> (a.k.a. lab demo) — 5-min ARM MMIO warm-up
Optional capstones	<code>lab 11</code> <code>lab 12</code> <code>lab 13</code> (nightly)
Verify install	<code>lab list && lab 01</code> → start → <code>sysbus.cpu PC</code> → quit
Edit + re-run	<code>edit ~/work/<lab>/...</code> , then <code>lab NN</code>
Read UART	(in the monitor) <code>sysbus.uartN CreateFileBackend @/tmp/uartN.log true</code> (in another terminal) <code>tail -f /tmp/uartN.log</code>
Quiet log spam	<code>logLevel 3 sysbus.uartN</code>
Pause/resume CPU	<code>pause / start</code>
Read PC	<code>sysbus.cpu PC</code>
Step one insn	<code>sysbus.cpu Step</code>
Reset firmware	<code>machine Reset</code>
Exit Renode	<code>quit</code>
Save your work	<code>gh repo fork --remote --remote-name=origin</code> (one-time)
	<code>git checkout -b my-experiments</code>
	<code>cp -ru ~/work/<lab>/ . /workspaces/renode-lab/labs/<lab>/</code> (per lab; NOT /labs/)
	<code>git add -A && git commit -m "..."</code> && <code>git push -u origin my-experiments</code>
Stop Codespace	bottom-left of VS Code → Stop codespace
Delete Codespace	<code>gh codespace delete -c <name> --force</code> (safe AFTER the push above)